

Apollo-RTOS: An AGC-Inspired Real-Time Operating System for Embedded Systems



M. Ahsan Al Mahir

Department of Computer Science
University of Oxford

Supervisor: Prof. Alex Rogers

A project report submitted for the degree of Master of Mathematics and Computer Science

Trinity 2025

9906 words

(using `texcount -inc -nobib -sum=1,1,0,1,0,0,0`)

Abstract

Embedded systems have advanced significantly, but many of the fundamental challenges encountered by early computing platforms remain relevant today. The Apollo Guidance Computer (AGC), operating with just a 2 MHz CPU, 4 KB of RAM, and 72 KB of ROM, introduced pioneering techniques in scheduling and fault tolerance—features critical to the success of mission-critical systems.

This project presents **Apollo-RTOS**, a real-time operating system for the `micro:bit v1`, inspired by the design principles of the AGC and reimaged through a modern lens. Apollo-RTOS adapts the AGC's hybrid cooperative-preemptive scheduling and recovery-oriented architecture while incorporating contemporary operating system features such as dynamic memory allocation, file systems on both flash and FRAM devices, inter-process synchronization, and a Unix-like shell interface. It supports up to 16 concurrent processes and is implemented in modern C++, leveraging RAII, templates, and type deduction for robustness and clarity.

Apollo-RTOS is built with a monolithic kernel design and includes an integrated testing framework for unit and subsystem tests. Through this work, the project demonstrates how historical embedded computing principles can be combined with modern programming paradigms to build resilient and efficient RTOS platforms for resource-constrained environments.

Contents

Contents	1
1 Introduction	3
1.1 Background & Motivation	3
1.2 Project Summary	3
2 The Apollo Guidance Computer	5
2.1 The Executive	6
2.1.1 The Waitlist	7
2.2 Restart Mechanism	8
2.2.1 The 1201 and 1202 Alarms During Apollo 11's Lunar Descent	9
2.3 Interpreter	9
2.4 Display and Keyboard	10
3 Inspiration behind Apollo-RTOS	11
4 Implementation Architecture and Design	13
4.1 Overview	13
5 Scheduling: Co-operative and Pre-emptive	15
5.1 Process Creation	16
5.2 Priority Modification	17
5.3 Context Switching	18
5.4 Process Suspension	19
5.5 Idle Process and Scheduler Initialization	20
5.6 Waitlist: Preemptive Scheduling	21
5.6.1 Wakeup Alarm	22
5.7 Synchronization Mechanism	23
5.7.1 Interrupts management	24
6 Memory Management and File System	26
6.1 File System	28
6.2 File system implementation	29
6.2.1 File system backend	29
6.2.2 File System Frontend	31
6.2.3 File system instantiation	33
7 Error Handling and Recovery	35

7.1	Boot Levels	35
7.1.1	How to Detect Boot Stages	36
7.2	Error handling	36
7.3	Recovery	38
7.3.1	Recovering a process	38
7.3.2	Recovering a task	40
7.4	Recovery at the startup	41
8	Drivers	43
8.1	Timers	43
8.1.1	Profiling	44
8.2	Serial	44
8.2.1	Listening on the UART	46
8.3	I2C	47
8.4	Display	50
8.5	Display	50
9	Shell and Shell Commands	51
9.1	Notable Shell Commands	52
10	Evaluation and Testing	54
10.1	Testing Framework	54
10.2	Evaluation of Apollo-RTOS	55
11	Conclusion	56
	References	57
A	Codebase Structure	58

§1 Introduction

1.1 Background & Motivation

Computing capabilities have grown at an unprecedented rate over the last century. The **Apollo Guidance Computer** (AGC), which played a critical role in the Apollo program, operated with a modest *2 MHz CPU, 4 KB of RAM, and 72 KB of ROM*. To maximize its limited resources, the AGC’s operating system introduced innovative solutions for *task scheduling, error handling, and resource-efficient computation*. One of its most groundbreaking features was a hybrid scheduling mechanism that combined cooperative and preemptive multiprogramming, ensuring that critical tasks could execute reliably even in adverse conditions. Additionally, the AGC incorporated a robust recovery mechanism, enabling fault tolerance in the face of system failures.

Today, a compact `micro:bit` device costing less than £20 possesses significantly more computing power than the AGC. Such microcontrollers are now ubiquitous, powering applications ranging from consumer electronics to mission-critical systems. Despite the vast improvements in computational power, many of the challenges that the AGC’s operating system addressed remain highly relevant, particularly in constrained embedded environments where real-time performance, reliability, and fault tolerance are essential.

1.2 Project Summary

This project investigates the design of the Apollo Guidance Computer (AGC) operating system and explores how its core ideas—most notably its hybrid cooperative-preemptive scheduling and fault-tolerant recovery—can be adapted to modern embedded systems. The result is **Apollo-RTOS**, a fully-featured real-time operating system implemented for the `micro:bit v1` device.

Apollo-RTOS faithfully adapts several AGC concepts such as priority-based cooperative multitasking, a preemptive task queue, and a system-wide recovery mechanism that tolerates resets and hardware faults. These are combined with modern operating system abstractions including dynamic memory allocation, file systems, inter-process communication, and a Unix-like shell interface. The implementation is written in modern C++, leveraging RAII, templates, and static typing for safety and performance.

At the architectural level, Apollo-RTOS follows a *monolithic kernel design*, incorporating a compact scheduler and a recovery-aware process model. It also includes a modular file system stack with support for both flash and FRAM-backed block devices, along with lightweight drivers for peripherals including UART, display, and I2C-based sensors. Developers interact with the OS using a shell and a suite of programmable applications, written against a familiar C++ API. An integrated testing framework supports unit and subsystem testing.

Key Contributions

- A lightweight, deterministic **hybrid scheduler** combining cooperative and preemptive se-

mantics. [§ 5]

- **Fault-tolerant process recovery** and boot-stage awareness using AGC-inspired restart groups. [§ 7]
- **Dual file system support** over flash and FRAM devices using a shared template-based backend. [§ 6]
- A minimalist **Unix-like shell interface** with support for foreground apps and command dispatch. [§ 9]
- A robust, modern **C++20 implementation** using RAII, templates, constexprs, and type deduction.
- A **unit and integration testing framework** to validate system components in isolation. [§ 10]

Apollo-RTOS demonstrates that historical system design choices, when reinterpreted through the lens of modern software engineering, can lead to elegant and effective solutions for contemporary embedded platforms. It provides a practical example of building robust, modular systems under tight resource constraints while remaining extensible, maintainable, and developer-friendly.

The source code of Apollo-RTOS can be found here:

<https://github.com/AnglyPascal/apollo-rtos>

§2 The Apollo Guidance Computer

The Apollo Guidance Computer (AGC) was an early digital computer developed in the 1960s to help navigate and control Apollo spacecraft on their missions to the Moon. It provided real-time guidance and control, making it essential for the success of the Apollo program.

The AGC ran at a speed of 2.048 MHz, which translated to about 40,000 instructions per second. The AGC had a 16-bit word length, 15 bits for data, and one parity bit. It had 2 kilowords or 4KB of erasable magnetic-core memory (RAM) along with 36 kilowords or 72KB of fixed core rope memory (ROM) for storing software and mission data. Astronauts interacted with the AGC using the Display and Keyboard (DSKY), a 21-digit display with a 19-button keyboard that allowed them to input commands and receive information. This interface was critical for making manual adjustments and monitoring the system during missions.



Figure 2.1: Margaret Hamilton standing next to listings of the software she and her MIT team produced for the Apollo Project.

Due to these constraints, the operating system, called the Executive, had to efficiently manage resources. It ensured that important tasks were prioritized over less critical ones and that key functions could continue even if the system experienced a reset due to an error.

The Executive was written in AGC assembly, where each instruction was 16 bits long. Because the instruction set was small, the system also included a sophisticated interpreter that acted as a virtual machine, providing more advanced pseudo-instructions beyond the basic AGC instructions.

The details of the AGC Executive explained below is taken from O'Brien [3].

2.1 The Executive

The central component of the AGC software was the Executive, which ensured fair allocation of system resources, such as CPU time and dynamic memory, to processes and tasks. To seamlessly perform real-time control and navigation tasks with extreme reliability and efficiency, the Executive implemented a **priority-based cooperative scheduling** algorithm, described below:

- The highest-priority process has exclusive access to the CPU.
- Lower-priority processes remain idle until the highest-priority process:
 - lowers its priority, or
 - goes to `sleep` while waiting for an event.
- If a new process with a higher priority is scheduled while another is running:
 - the new process waits until the current process lowers its priority or goes to `sleep`,
 - then the new process starts executing.

Core Sets Information about running processes was stored in a table called Core Sets, which functioned similarly to modern process descriptors. Each Core Set had a structure similar to:

```

1 struct CORE_SET {
2     word priority;
3     word program_counter;
4     word bbank_reg; // area in unswitched memory where the code is
5     word vac_ptr;   // pointer to extra memory
6
7     word mpac[7];   // multi-purpose accumulators
8     word mpac_mode;
9 };

```

Each active process occupied one Core Set. The system provided:

- 6 Core Sets in the Command Module, and
- 7 Core Sets in the Lunar Module.

In either case, Core Set 0 always contained the currently running process. A context switch was essentially performed by swapping the current Core Set with Core Set 0.

Addressable Memory The 7 `mpac` words served two purposes. In normal mode, they provided 7 words of freely usable memory. In interpreter mode, they functioned as accumulator registers, storing values for calculations performed by the interpreter. If a process required more memory, it could request 42 additional words from the **Vector Accumulator** (VAC) area.

Starting a New Process To start a new process, an empty Core Set had to be located, followed by a VAC area if needed. After populating the Core Set, the Executive determined if the new process had a higher priority.

A special register, **NEWJOB**, was used for this purpose:

- If **NEWJOB** is 0, the current process has the highest priority.
- If **NEWJOB** is non-zero (say d), the process represented by Core Set d has the highest priority.

When a new process was created, its priority was compared to that of the currently running process, and **NEWJOB** was updated accordingly.

Process Switching A context switch only occurred when the current process voluntarily relinquished control. The process was required to check **NEWJOB** at least every 20ms to detect higher-priority processes. If **NEWJOB** was not checked for over 640ms, it was assumed that the process had stalled, triggering a hardware restart.

To switch processes, the Executive stored the current context in Core Set 0 and swapped it with the Core Set indexed by **NEWJOB**. Since the program counter was restored directly from the Core Set, execution of the switched-in process resumed automatically from its last instruction.

If the current process was still the highest-priority process but needed to go to sleep or lower its priority, the system scanned the Core Set table to identify the next highest-priority process and performed a switch accordingly.

Sleep and Wakeup A process could voluntarily relinquish execution while waiting for an external event by going to sleep. The **JOBSLEEP** instruction marked a process as sleeping by negating its priority. The next highest-priority process was then scheduled.

When the awaited event occurred, another process could wake the sleeping process using the **JOBWAKE** instruction. This reversed the priority negation, making the process runnable again. **JOBWAKE** then checked **NEWJOB** to determine if the woken process should run immediately.

A process could also call **DELAYJOB** to sleep for a fixed duration. This scheduled an "alarm" task via the **waitlist**, described in the next section.

2.1.1 The Waitlist

The core processes in the Executive were all cooperative, meaning that even the highest-priority process would not preempt the currently running process until it voluntarily relinquished execution. The waitlist introduced preemptive scheduling capabilities to the AGC, allowing tasks to be scheduled with strict deadlines. Every 10ms, a timer interrupt was triggered to execute any waitlist task that was due.

In essence, the waitlist was a *sorted linked list* of tasks ordered by increasing deadlines. When a new waitlist task t was scheduled with a particular deadline, it was inserted into the list such that:

- any task t_0 with an earlier deadline appeared before t , and

- any task t_1 with a later deadline appeared after t .

Every 10ms, the timer interrupt checked whether the first task in the list had reached its deadline. If so, the task was executed and then removed from the list. Since waitlist tasks were executed within an interrupt handler, they had to be short-lived—typically around 5ms or 150–200 instructions.

The waitlist implementation in Apollo-RTOS closely follows that of the AGC. Implementation details will be discussed later.

2.2 Restart Mechanism

Despite its limited resources, the AGC needed to be highly reliable, even in the presence of errors. Modern CPU and OS designs dedicate significant resources to detecting and recovering from faults. However, the AGC lacked the hardware to support sophisticated error handling. Furthermore, without a memory protection unit, it was nearly impossible to reliably detect errors caused by memory corruption. As a result, the AGC adopted a radical approach:

Just restart everything.

However, restarting at an arbitrary point in time required careful handling, as it could terminate critical tasks mid-execution.

To address this, critical tasks and processes could define recovery behavior in case of a restart. Each program belonged to one of six restart groups. A process could use the `PHASCHNG` instruction to set up an entry in the restart phase table. Each group could recover only one process at a time. Based on its needs, a process could configure its group's recovery procedure to one of the following four modes:

- **Inhibit group restart:** No processes in this group are restarted.
- **Restart last display:** Restore and display the last known contents before the restart.
- **Execute one restart routine:** Restart the process from the beginning with minimal cleanup.
- **Execute two restart routines:** Schedule a task to clean up data and re-enter the process from a designated point.

To conserve memory, the AGC stored recovery information in a compressed format within the phase tables.

During a restart:

- All active jobs were canceled, and all tasks in the waitlist queue were deleted.
- Interrupts were disabled, and all tables were cleared and reinitialized.
- The integrity of the restart phase table was verified.
- Restart processing was carried out sequentially for each active phase in groups 6 to 1.

These mechanisms ensured that the AGC could gracefully recover from many types of fail-

ures—an essential requirement for spacecraft operations, where computer failures could have catastrophic consequences.

2.2.1 The 1201 and 1202 Alarms During Apollo 11's Lunar Descent

During Apollo 11's final descent to the lunar surface, the Lunar Module's guidance computer triggered several unexpected alarms—specifically, program alarms 1201 and 1202. These alarms indicated “executive overflows,” meaning the computer's task scheduling system was overloaded and unable to complete all its tasks in real-time.

The root cause was traced to the rendezvous radar being left on during descent. This radar, intended for docking procedures, inadvertently fed spurious data to the computer due to a hardware design quirk [7]. This resulted in excessive “cycle stealing,” where the radar's data processing consumed significant CPU time, leaving insufficient resources for other tasks. Compounding the issue, an additional command to display altitude data further taxed the system, leading to the alarms.

Despite these alarms, the guidance computer's design allowed it to prioritize critical tasks. Upon encountering an overload, the system would perform a soft restart, shedding lower-priority tasks while maintaining essential functions like engine control and navigation. This resilience was a testament to the foresight of the software engineers, particularly Hal Laning, whose design ensured that even in the face of unexpected issues, the mission could proceed safely.

Mission Control, relying on their understanding of the system's behavior, assessed the situation and determined it was safe to continue. Their decision, combined with the computer's robust design, allowed Apollo 11 to successfully land on the Moon, turning a potential mission abort into a historic achievement.

2.3 Interpreter

The Apollo Guidance Computer (AGC) featured a custom virtual machine known as the **Interpreter**, designed to extend its limited instruction set. This was necessary because the AGC's native instruction set was highly restricted, containing only 11 opcodes in AGC Block II. The Interpreter enabled more complex operations while conserving memory and improving execution efficiency for key tasks.

The Interpreter functioned as a *bytecode interpreter*, running on top of the AGC's native instruction set. It allowed the execution of *pseudo-instructions*, which were more powerful than basic machine instructions. These pseudo-instructions were stored as compact *6-bit opcodes*, maximizing memory efficiency.

The AGC Interpreter provided an extended set of *arithmetic and vector operations*, primarily aiding navigation and guidance computations. Key capabilities included:

- **Arithmetic Operations:** Fixed, single, and double-precision floating-point arithmetic, as well as `sqrt` calculations.

- **Vector and Matrix Operations:** Dot products, cross products, and matrix transformations.
- **Trigonometric Functions:** Sine, cosine, and related functions for celestial navigation.
- **Data Movement and Manipulation:** Indexed memory access (not available in native AGC instructions), indirect addressing, and array operations.
- **Flow Control:** Conditional branching and subroutine-like operations for reusable code execution.

The AGC’s native instruction set prioritized speed but lacked flexibility for advanced navigation tasks. The Interpreter enabled *higher-level mathematical operations* to be executed compactly and efficiently.

2.4 Display and Keyboard

The AGC featured a unique human-machine interface known as the *Display and Keyboard (DSKY)*, allowing astronauts to communicate with the computer using an intuitive *verb-noun paradigm*. With its green *electroluminescent displays*, *19-key input panel*, and *status indicators*, the DSKY played a crucial role in both the Command Module and Lunar Module cockpits. Despite its simple appearance, it incorporated sophisticated software routines, enabling astronauts to *monitor spacecraft systems, compute trajectories, and execute maneuvers with precision*.

The *verb-noun paradigm* structured commands similarly to natural language, where astronauts first selected a *verb* (the action) followed by a *noun* (the object of that action). Using *two-digit numeric codes* for both components, astronauts could issue commands efficiently. For example, the command “*Verb 06 Noun 36*” instructed the AGC to *display the current time*. This system provided a consistent interface for a range of operations, from *simple data retrieval to complex trajectory calculations*.



Figure 2.2: DSKY interface diagram by Oona Räisänen

§3 Inspiration behind Apollo-RTOS

The real-time operating system developed in this project, Apollo-RTOS, draws inspiration from both the Apollo Guidance Computer (AGC) operating system and the modern Linux OS. While the scheduler and error handling concepts are derived from the AGC, the naming conventions, user interface, and implementation are influenced by Linux.

The AGC OS was designed to meet specific mission requirements, allowing for a simplified and optimized design. In contrast, Linux is a general-purpose OS, designed to support applications with vastly different needs, making its architecture more complex.

The AGC operated within a well-defined hardware and software environment, where all components could be trusted. As a result, isolating the hardware, OS, and application code was unnecessary. Additionally, the AGC lacked memory protection features, making kernel/user space isolation, a fundamental aspect of general-purpose operating systems like Linux, impossible.

In Apollo-RTOS, we adopted the same high-level design principles as the AGC. Our goal was to implement its key innovations in a modern context while ensuring the OS could function efficiently in a resource-constrained environment. To achieve this, we made the following design choices:

- **Hardware Selection:** The `micro:bit v1` was chosen as the primary target platform. Its Cortex-M0 CPU is one of the smallest in the ARM family and closely resembles the AGC hardware. Additionally, it includes pre-installed peripheral devices, which serve as useful test cases for OS functionality.
- **Monolithic Kernel Design:** Inspired by Linux, Apollo-RTOS follows a monolithic kernel architecture due to its simpler implementation and better execution efficiency in a constrained environment. However, there's a significant trade-off: with a monolithic kernel, verifying and testing the OS is more difficult.
- **No Kernel/User Space Isolation:** In microcontroller programming, the entire application is compiled into a single binary, meaning the kernel and user space are inherently combined. Since the entire codebase has full access to the kernel internals, enforcing strict isolation would add unnecessary complexity.
- **Library Calls Instead of System Calls:** Traditional system calls are beneficial only when user and kernel space are strictly separated. Since Apollo-RTOS does not enforce such isolation, we opted for direct library function calls, significantly simplifying implementation.
- **Shared Memory for Inter-Process Communication (IPC):** While shared memory introduces risks like race conditions and synchronization issues, it remains the fastest and simplest IPC method, making it the preferred choice for Apollo-RTOS.
- **Liberty to Use Memory Liberally:** While the `micro:bit` has limited memory, it still offers significantly more than the AGC. As Donald Knuth famously said, "premature optimization is the root of all evil," our priority is a functional and maintainable implementation rather than aggressively minimizing memory usage.

Additionally, we drew inspiration from the `microbian` OS for the `micro:bit`, as developed by Spivey [6]. Specifically, we referenced its hardware description headers, startup code, and interrupt handler implementations in certain parts of our design.

For our language and development ecosystem, we chose to work with C++ for several reasons:

- C++ reduces boilerplate code compared to C with its concise syntax for object creation, templates, and RAII paradigm.
- The C++ compiler provides extensive optimizations through templates, `constexpr` variables and functions, and references.
- C++ templates and type deduction allow for cleaner, more expressive code, improving development experience.
- And finally, a personal preference: *I just wanted to learn more C++.*

§4 Implementation Architecture and Design

Apollo-RTOS is a real-time operating system inspired by the AGC, designed for the `micro:bit v1` device. It follows a monolithic kernel architecture and is built around a cooperative scheduler with preemptive capabilities.

This section provides an overview of the core modules in Apollo-RTOS and their interconnections, followed by a discussion of the codebase structure.

4.1 Overview

The major modules of Apollo-RTOS are:

Scheduler The core of the OS, responsible for:

- Managing process execution, including creation, termination, priority changes, pausing, and resuming.
- Handling context switching between processes.

Waitlist Implements a deadline-driven task scheduler, similar to cron jobs, enabling time-sensitive operations to be executed at precise intervals.

Signals and Mutexes Enable low-overhead inter-process communication (IPC) and synchronization mechanisms among the processes.

Memory Manager Handles all dynamic memory allocation. It implements a freelist-based `malloc` for allocating and deallocating runtime memory and manages process stack allocations.

File System Provides persistent storage, implemented using an external FRAM over I2C. An optional Flash-based file system can also be enabled.

Debugging and Error Handling Provides essential debugging utilities, including:

- `debug`: Outputs diagnostic information.
- `assert`: Handles runtime error detection.

Recovery Ensures system stability by protecting critical processes and tasks from various resets and failures.

Drivers Interface with the hardware components of the `micro:bit`. The key drivers include:

- **Timer**: Apollo-RTOS utilizes two 16-bit timers—`Timer1` as a system clock and `Timer2` for profiling.

- **Serial:** Manages UART communication.
- **I2C:** Handles communication with the I2C master on the `micro:bit`.
- **Display:** Controls the 5×5 LED matrix on the board.

Shell Implements a Unix-like command-line interface that allows users to manage and debug processes over UART. It maintains a list of available commands mapped to various processes.

§5 Scheduling: Co-operative and Pre-emptive

The core scheduler of Apollo-RTOS is a priority-based cooperative scheduler. It allows the highest-priority process to run indefinitely until it voluntarily gives up execution, at which point other processes are given the opportunity to run. This simple approach offers several benefits:

- **Simplicity:** The implementation is straightforward.
- **Fewer race conditions:** Since one process never preempts another, data races on shared variables are largely eliminated.

However, processes must be aware of their runtime context and yield execution periodically to prevent deadlocks. This can be done in two ways:

- Lowering their priority.
- Going to sleep.

With this design, the interface for process creation and management remains simple:

```
1 // Register a process defined by 'proc_def', passing 'param' as an
   argument
2 pid_t reg_proc(const proc_def_t *proc_def, void *param);
3
4 // Increase the priority
5 void incr_priority(pid_t pid, priority_t priority);
6
7 // Decrease the priority
8 void decr_priority(priority_t priority);
```

Defining a process is minimalistic:

```
1 struct proc_def_t {
2     string      name;
3     priority_t  priority;
4     size_t      stk_sz;
5     runnable_t  func;          // where runnable_t = void ()(void);
6 };
```

At runtime, each process is represented in a process descriptor table, which stores essential metadata about active processes. The most important fields are:

```
1 struct proc_t {
2     priority_t  priority;      // Current priority
3     byte_t      *stk_ptr;      // Current stack pointer
4     byte_t      *stack;        // Start of the stack area
5     size_t      stk_sz;        // Stack size
6     void        *param;        // Parameter passed to the process function
7     // ... other fields
8 };
```

Stack Layout in Arm Cortex-M0

To understand the scheduling mechanism, we first examine the stack layout in the Arm Cortex-M0 architecture:

The stack in Arm architecture starts at a higher address and grows downward.

When execution is preempted by an interrupt, the hardware automatically pushes key registers onto the stack in the following order (from lower to higher addresses):

```

new stack pointer -> |
                    | r0
                    | r1
                    | r2
                    | r3
stack grows      ^   |
downwards        |   | r12
                    | lr   Link register
                    | pc   Program counter
old stack pointer -> | psr   Status register

```

To enable context switching between processes, additional registers must be saved manually. This is done via software, giving us control over the order. We chose the following layout:

```

new stack pointer -> |
                    | r8
                    | r9
                    | r10
                    | r11
                    | r4
                    | r5
                    | r6
stack pointer after | r7
hardware pushes regs -> | lr_intr   Return from interrupt address

```

With the stack management details covered, we can now discuss how the scheduling algorithm works.

5.1 Process Creation

The implementation of **reg_proc** can be summarized as follows:

```

src/core/sched.cpp
1 pid_t reg_proc(const proc_def_t *def, void *param)
2 {
3     auto [name, priority, stk_sz, proc_func] = *def;
4     proc_t *proc = procs.alloc(); // allocate a process descriptor
5     proc->param = param; // store the param for future cleanup
6
7     stack_allocator.acquire(proc, stk_sz, proc_func, param, exit);
8     incr_priority(pid, priority);
9
10    // ...
11    return pid;

```

12 }

The key operations occur on lines 7 and 8. In line 7, the local pool allocator assigns a block of memory of size `stk_sz` and initializes it with the required parameters.

A newly created process is treated as if it has just undergone a context switch. This means the stack must be set up with the appropriate register values in the correct order, as discussed earlier. Line 7 ensures this setup is performed correctly.

Examining the `acquire` function:

```

src/core/stack_allocator.h
1 void acquire(proc_t *proc, size_t stk_sz, runnable_t func, void *param,
2             void (*ret)())
3 {
4     proc->stack = (byte_t *)pool.alloc(stk_sz); // allocate the stack
5     // ...
6
7     auto stk_ptr = (context_t *) (proc->stack + stk_sz - sizeof(context_t));
8
9     // setup initial stack frame
10    stk_ptr->psr    = init_psr;           // 0x01000000 to enable Thumb mode
11    stk_ptr->pc     = (uint32_t)func;      // entry to the process
12    stk_ptr->lr     = (uint32_t)ret;       // exit from the process
13    stk_ptr->r0     = (uint32_t)param;     // parameter
14    stk_ptr->lr_intr = lr_intr_magic;      // return address from interrupts
15
16    proc->stk_ptr = (byte_t *)stk_ptr;
17    proc->stk_sz = stk_sz;
18 }

```

Lines 10-14 populate the process context. The entry point is stored in the pc register, the exit sequence in the lr register, and the process parameter in r0. The interrupt return address is also set.

5.2 Priority Modification

A singleton object, `cpu`, tracks the currently running process and the highest-priority process:

```

src/core/sched.cpp
1 volatile struct {
2     proc_t *hi_proc = nullptr; // highest priority process
3     proc_t *curr_proc = nullptr; // currently running process
4 } cpu;

```

This object functions similarly to the `NEWJOB` register in the AGC. When a process is created, its priority is compared with the highest-priority process. If it is higher, `hi_proc` is updated.

In `reg_proc` (line 8), `incr_priority` is called:

```

src/core/sched.cpp
1 void incr_priority(pid_t pid, priority_t priority)
2 {
3     proc_t *proc = procs[pid];
4     // ... disable interrupts
5     proc->priority = priority; // set priority
6     if (cpu.hi_proc == nullptr || cpu.hi_proc->priority < priority)
7         cpu.hi_proc = proc;
8 }

```

When a high-priority process completes its critical section, it should lower its priority to allow other processes to run:

```

src/core/sched.cpp
1 void decr_priority(priority_t priority)
2 {
3     cpu.curr_proc->priority = priority;
4     cpu.hi_proc = procs.max_priority();
5     change_proc();
6 }

```

The highest-priority process is updated, and **change_proc** is called to check if a context switch is needed:

```

src/core/sched.cpp
1 void change_proc()
2 {
3     // enable interrupts
4     if (cpu.hi_proc != cpu.curr_proc)
5         reschedule(); // triggers pendsv interrupt
6 }

```

Now, we examine how context switching is performed.

5.3 Context Switching

When `reschedule()` triggers the `pendsv` interrupt, the hardware first pushes key registers onto the stack. Then, `pendsv_handler` is called. We then save additional registers, swap stack pointers, and restore the state for the new process.

The register saving/restoring is handled in assembly, while stack pointer swapping is done in C++:

```

src/mpx.s
1 @@@ push software-saved registers onto stack
2 .macro isave
3     push {r4-r7, lr}           @ Copy from high to low
4     mov r4, r8
5     mov r5, r9
6     mov r6, r10
7     mov r7, r11

```

```

8  push {r4-r7}
9  mrs r0, msp                                @ Copy from high to low
10 .endm
11
12 @@@ pop software-saved registers
13 .macro irestore                            @ Expect process sp in r0
14 msr msp, r0
15 pop {r4-r7}
16 mov r11, r7
17 mov r10, r6
18 mov r9, r5
19 mov r8, r4
20 pop {r4-r7}
21 pop {pc}                                @ pop lr into pc, jumping to new process
22 .endm
23
24 @@@ PendSV handler for context switching
25 .global pendsv_handler
26 .thumb_func
27 pendsv_handler:
28     isave                                @ Save current process state
29     bl cxt_switch                        @ Select new process
30     irestore                            @ Restore new process state

```

The definition of `cxt_switch`:

```

src/core/sched.cpp
1 void *cxt_switch(void stk_ptr)
2 {
3     if (/* cpu.curr_proc is done */)
4         /* release resources */
5     else
6         cpu.curr_proc->stk_ptr = (byte_t *)stk_ptr; // store old stk_ptr
7
8     cpu.curr_proc = cpu.hi_proc;
9     return cpu.hi_proc->stk_ptr; // restore new stk_ptr
10 }

```

5.4 Process Suspension

A process can voluntarily yield execution either by lowering its priority or by suspending execution (sleeping) to be woken up later.

The wake-up mechanism is more involved, so we will discuss it in two parts. A process can wake up in one of two ways:

- By setting an alarm when going to sleep, which wakes it up after a specified time.
- By waiting for a specific event to occur before resuming execution.

The first method relies on the waitlist, while the second requires a synchronization mechanism. We will first discuss process suspension before covering these mechanisms.

A runnable process is represented by a positive priority value. To put a process to sleep while preserving its original priority, we simply negate its priority.

src/core/sched.cpp

```

1 void sleep(time_t period = 0)
2 {
3     if (period > 0)
4         /* set an alarm in the waitlist */
5         /* else suspend execution until another process wakes me up */
6
7     // Negate priority to mark process as ASLEEP
8     decr_priority(-cpu.curr_proc->priority);
9 }

```

A process that wants to wake up a sleeping process simply negates its priority:

src/core/sched.cpp

```

1 void wakeup(pid_t pid) {
2     incr_priority(pid, -procs[pid]->priority);
3 }

```

5.5 Idle Process and Scheduler Initialization

Since it could happen that all the running processes are waiting for some event to occur, when they are all sleeping, something needs to keep running in the cpu. That's the job for the IDLE_PROC. This is the lowest priority process defined when the scheduler is initialized during startup that basically just loops

src/core/sched.cpp

```

1 void idle_task(void *)
2 {
3     intr_enable(); // re-enable interrupts after setup is done
4     while (true)
5         change_proc();
6     // idle process always checks if any other process is ready to run
7 }
8
9 void sched::init()
10 {
11     intr_disable(); // disable interrupt while setup is ongoing
12
13     IDLE_PID = reg_proc(&idle_proc_def, nullptr);
14     cpu.curr_proc = procs[IDLE_PID];
15
16     // ... set up rest of the processes
17
18     cpu.curr_proc->stk_ptr = /* setup the stack for the idle_task */;
19     /* setup the assigned stack and jump directly to idle_task */
20     __run(idle_task, &cpu.curr_proc->stk_ptr);
21 }

```

`__run` is an assembly function, that just loads the stack pointer to the `sp` register, and jumps to the function provided:

```

1  @@@ enter process mode
2      .global __run
3      .thumb_func
4  __run:
5      ldr r2, [r1]                @ load idle proc's stack value
6      msr msp, r2
7      isb                        @ Drain the pipeline
8      bx r0                      @ Call the body

```

The `sched::init` function needs to do some other initialization as well to setup the processes that will be running after the startup, and we'll discuss them in § 7.4.

5.6 Waitlist: Preemptive Scheduling

The waitlist is implemented as a sorted linked list of tasks:

```

                                src/core/waitlist.cpp
1  struct waitlist_t {
2      time_t remaining = 0;
3      runnable_t func = nullptr;
4      void *param = nullptr;
5      waitlist_t *next = nullptr; // Linked list link
6  };

```

If a task t_0 is scheduled to run in n_0 ticks, it is placed in the waitlist between tasks t_1 and t_2 such that if t_1 and t_2 have deadlines n_1 and n_2 , then $n_1 \leq n_0 \leq n_2$. We set:

$$\text{remaining} = n_0 - n_1$$

Thus, the time until executes is the sum of the `remaining` values for all preceding tasks.

This algorithm, originally used in the AGC, has key implications:

- Maintaining a sorted linked list has insertion complexity. However, with an upper limit of `N_WAITLIST = 16`, this cost is negligible.
- Using remaining values, we only need to check the first task on each timer tick, reducing computational overhead.

Task registration and execution are implemented as follows:

```

                                src/core/waitlist.cpp
1  void reg(string name, time_t interval, runnable_t func, void *param)
2  {
3      waitlist_t *task = alloc();
4      waitlist_t *hd = &waitlist_hd;
5      time_t prev = 0;
6      while (hd->next != nullptr && prev + hd->next->remaining <= interval) {
7          prev += hd->next->remaining;

```

```

8     hd = hd->next;
9 }
10 time_t remaining = interval - prev;
11
12 if (hd->next != nullptr)
13     hd->next->remaining -= remaining;
14
15 *task = {name, remaining, func, param, hd->next};
16 hd->next = task;
17 }

```

During a timer interrupt, expired tasks are executed:

```

src/core/waitlist.cpp
1 void run()
2 {
3     auto head = &waitlist_hd;
4     if (head->next != nullptr)
5         head->next->remaining -= update_interval;
6
7     while (head->next != nullptr && head->next->remaining == 0) {
8         auto task = head->next;
9         head->next = task->next;
10
11         auto [name, remaining, func, param, next] = *task;
12         dealloc(task);
13         func(param);
14     }
15 }

```

This approach minimizes overhead in interrupt-inhibited situations, making it ideal for real-time constraints.

5.6.1 Wakeup Alarm

With the waitlist implemented, setting up an alarm is simple:

```

src/core/sched.cpp
1 void sleep(time_t period = 0)
2 {
3     auto proc = cpu.curr_proc;
4
5     auto default_alarm = [](void *ptr) {
6         auto proc = (proc_t *)ptr;
7         incr_priority(/ pid of proc */, -proc->priority);
8     };
9
10    if (period > 0)
11        waitlist::reg(proc->name, period, default_alarm, (void *)proc);
12
13    decr_priority(-proc->priority);
14 }

```

5.7 Synchronization Mechanism

We implement channels for synchronization, using a FIFO queue of waiting processes based on `circular_buffer`:

```

include/core/sched.h
1 template <size_t chan_len>
2 struct chan_t : public circular_buffer<pid_t, chan_len> {};

```

A process waiting for an event enqueues itself and sleeps, while another process signals the event by dequeuing and waking it up:

```

include/core/sched.h
1 template <size_t chan_len>
2 void wait(chan_t<chan_len> &chan)
3 {
4     intr_guard guard;
5     chan.enqueue(/* current process */);
6     sleep();
7 }
8
9 template <size_t chan_len>
10 void notify(chan_t<chan_len> &chan)
11 {
12     intr_guard guard;
13     if (chan.empty())
14         return;
15     wakeup(chan.dequeue());
16 }
17
18 template <size_t chan_len>
19 void notify_all(chan_t<chan_len> &chan)
20 {
21     intr_guard guard;
22     while (!chan.empty())
23         wakeup(chan.dequeue());
24 }

```

Using channels and `wait`, `notify`, we can implement a `mutex`

```

include/utility/mutex.h
1 template <size_t chan_len>
2 class mutex {
3     chan_t<chan_len> lock_chan;
4     bool busy = false;
5
6 public:
7     void lock()
8     {
9         while (busy)
10             sched::wait(lock_chan);
11         busy = true;
12     }

```

```

13
14 void unlock()
15 {
16     busy = false;
17     sched::notify(lock_chan);
18 }
19 };

```

Usually we acquire a mutex before a critical section, and then release it at the end of the section. Using the RAII paradigm, we can implement a class `lock_guard` (named after `std::lock_guard`) that acquire the mutex upon construction, and releases it in its destructor, giving a simple interface for locking.

```

include/utility/mutex.h
1 template <typename _mutex>
2 class lock_guard {
3     _mutex &_m;
4
5 public:
6     lock_guard(_mutex &m) : _m{m} { _m.lock(); }
7     ~lock_guard() { _m.unlock(); }
8 };

```

5.7.1 Interrupts management

We also implement a similar mechanism for disabling and enabling interrupts before and after a critical section. This implementation uses template type deduction to choose between the two scenarios when we want to disable all interrupts vs disable only one interrupt:

```

include/core/irq.h
1 template <typename T>
2 class intr_guard
3 {
4     int irq;
5
6 public:
7     // when no argument is passed, deduce T = void & disable all interrupts
8     intr_guard(void) requires std::same_as<T, void>
9         : irq{get_primask()}
10    {
11        intr_disable();
12    }
13
14    // re-enable all interrupts if T = void
15    ~intr_guard() requires std::same_as<T, void>
16    {
17        set_primask(irq);
18    }
19
20
21    // if an argument is passed, deduce T = int, and disable only that irq

```

```

22 intr_guard(int irq) requires std::same_as<T, int>
23   : irq{irq}
24   {
25       disable_irq(irq);
26   }
27
28
29   // if T = int, enable only that irq
30   ~intr_guard() requires std::same_as<T, int>
31   {
32       clear_pending(irq);
33       enable_irq(irq);
34   }
35 };
36
37 // type deduction hints for the constructor
38 intr_guard(void) -> intr_guard<void>;
39 intr_guard(int) -> intr_guard<int>;

```

This interface is then used simply as:

```

1 { // critical section
2   intr_guard guard; // disable all interrupts and re-enable at the end
3 }
4
5 { // critical section
6   intr_guard guard{UART_IRQ}; // disable only the UART interrupt
7 }

```

§6 Memory Management and File System

A central part of Apollo-RTOS is its memory allocation and management system. All run-time allocation is handled through instances of a pool allocator, which provides a simple and efficient form of dynamic memory management.

- The allocator maintains a linked list of free memory chunks.
- On an allocation request, it searches the free list for a chunk of sufficient size.
- If a suitable chunk is found, it is returned; otherwise, more memory is requested from the global allocator.
- When **free** is called, the chunk is returned to the free list.

```

                                include/utility/allocator.h
1  template <byte_t *(*alloc_func)(size_t), size_t alignment>
2  class allocator
3  {
4      chunk_t free_hd;
5      static constexpr size_t header_sz = sizeof(chunk_t);
6
7  public:
8      // allocate chunk of sz bytes
9      byte_t *alloc(size_t sz)
10     {
11         sz = roundup(sz, alignment);
12         auto chnk_sz = header_sz + sz;
13
14         chunk_t *chunk = /* find a chunk in free_hd, nullptr otherwise */;
15         if (chunk == nullptr) {
16             chunk = (chunk_t *)alloc_func(chnk_sz); // get new chunk
17             chunk->sz = sz;
18         } else {
19             /* remove chunk from free_hd */
20         }
21
22         return (byte_t *)chunk + header_sz;
23     }
24
25     // return chunk allocated by this allocator
26     void dealloc(byte_t *ptr)
27     {
28         if (ptr == nullptr)
29             return;
30         auto chunk = (chunk_t *) (ptr - header_sz);
31         /* insert chunk to free_hd */
32     }
33 };

```

Having an instantiable allocator instead of a global **malloc** function has key benefits:

- Since the hardware lacks a Memory Protection Unit (MPU), this helps isolate unrelated

memory allocations in software, reducing memory corruption risks.

- It provides a convenient interface for creating local pool allocators.

Using this allocator, we implement kernel-space memory management via **kmalloc** and **kfree**.

```
src/core/memory.cpp
1 namespace kmem
2 {
3     allocator<alloc_heap, 8> kpool;
4
5     void *kmalloc(size_t sz) { return kpool.alloc(sz); }
6
7     void kfree(void *ptr)
8     {
9         if (ptr == nullptr)
10             return;
11         kpool.dealloc((byte_t *)ptr);
12     }
13 } // namespace kmem
```

For user-space memory allocation, the implementation is more involved. When a process allocates memory, it is expected to free it explicitly. However, if the process encounters an error or is terminated prematurely, the scheduler must clean up any unfreed memory.

Each process descriptor maintains a list of allocated memory chunks. When a process terminates, all remaining allocations in this list are freed automatically.

```
src/core/memory.cpp
1 namespace heap
2 {
3     allocator<alloc_heap, 8> pool;
4
5     void *malloc(size_t sz)
6     {
7         byte_t *ptr = pool.alloc(sz);
8         /* insert chunk to the current process' allocated list */
9         return ptr;
10    }
11
12    void free(void *ptr)
13    {
14        if (ptr == nullptr)
15            return;
16        /* remove chunk from process' allocated list */
17        pool.dealloc((byte_t *)ptr);
18    }
19
20    // called at process teardown
21    void cleanup(chunk_t *hd)
22    {
23        while (hd->next != nullptr) {
```

```

24     auto ptr = (byte_t *)hd->next + sizeof(chunk_t);
25     /* remove hd->next from the list */
26     pool.dealloc(ptr);
27 }
28 }
29 } // namespace heap

```

6.1 File System

Flash The `micro:bit` features approximately 256KB of flash memory. A build of Apollo-RTOS with the `-O2` flag produces a binary of about 92KB, while using `-Os` reduces it to 76KB. In either case, there is ample remaining space in flash that can be used for long-term storage. However, flash memory has significant drawbacks:

- **Inefficient writes:** Writing to flash requires first erasing entire 1KB pages, taking about **22.3ms** per page. Writing individual bytes afterward takes **20μs** each [4].
- **Limited endurance:** The `micro:bit`'s flash memory has an endurance of only **20,000 erase cycles** and retains data for approximately **10 years** under normal conditions. Excessive writes degrade reliability.

Despite these limitations, reading from flash is fast since the CPU has direct access to it—after all, the program itself runs from flash. This makes flash memory suitable for read-heavy, infrequent-write storage needs.

FRAM Ferroelectric RAM (FRAM) addresses flash memory's shortcomings. Unlike flash, FRAM is non-volatile, with significantly higher endurance and retention rates. The MB85RC2567 FRAM chip [2] used in Apollo-RTOS provides:

- **10¹² erase cycles** – far exceeding flash longevity.
- **95+ years** of data retention under normal conditions.
- **32KB capacity**, operating at 1MHz I2C, though we use 100KHz.

However, FRAM has its own trade-offs:

- **I2C communication overhead:** Since the `micro:bit` lacks integrated FRAM, it is connected as an external I2C peripheral. This introduces bus contention, affecting read/write performance.
- **Slower reads than flash:** Unlike flash, which the CPU can access directly, FRAM requires I2C transactions, making reads slower. However, writes are significantly faster than flash.

Since both storage options have their strengths and weaknesses, Apollo-RTOS implements file systems on both. Instead of maintaining two separate file systems, we designed an abstract block device that can be instantiated with different parameters for each storage medium.

A block device is characterized by:

- Constants defining block size, total blocks, etc.

- Read/Write methods for interacting with the device.

Since these constants are frequently used in file system operations, we implemented them as template parameters rather than function parameters. This allows the compiler to optimize operations more effectively.

Thus, our file system implementation is entirely header-only, residing in the `include/fs/` directory.

6.2 File system implementation

The file system consists of two layers of tables with extensive bookkeeping. Below, we outline key concepts and methods.

fs_desc_t defines the file system parameters. Two instances—one for flash, another for FRAM—are constructed at compile time and passed as template parameters to the implementation.

```

include/fs/fs_desc.h
1 template <...>
2 struct fs_desc_t {
3     using addr_t = /* type of address supported by device */;
4     // for flash, it is uint32_t since writes must be word aligned
5     // for fram, it can be uint16_t to save space
6
7     size_t blk_sz; /* size of a block */
8     // for flash, it is 1024,
9     // since that's the smallest unit we can effectively write
10    // for FRAM, it is 128
11
12    size_t max_num_blks; /* max number of blocks per file */
13    size_t n_inodes; /* number of files */
14
15    size_t n_open_files; /* number of open files */
16
17    // methods to transfer data to and from the target device
18    void (*write)(addr_t addr, uint8_t *buf, size_t buf_sz);
19    void (*read)(addr_t addr, uint8_t *buf, size_t buf_sz);
20 };

```

6.2.1 File system backend

Each file is represented by an **inode_t**, inspired by Linux inodes, containing file metadata.

```

include/fs/fs_bck.h
1 template <typename desc_t, desc_t desc>
2 struct inode_t {
3     using addr_t = typename desc_t::addr_t;
4     using blk_addr_t = uint8_t;

```

```

5     // in either flash and fram, we can refer to a block by a 8 bit uint
6
7     size_t nblks; /* number of blocks in this file */
8     blk_addr_t blks[desc.max_num_blks]; /* blocks in this file */
9
10    uint32_t flag; /* stats encoded into a single uint32_t */
11 };

```

We use `fn_t = uint8_t` as the file name. We currently don't map file name strings to the corresponding inode, and directly refer the file by the inode number.

The file system header maintains:

- A table of inodes.
- A **bitset** tracking free and occupied blocks.

```

                                include/fs/fs_bck.h


---


1 template <typename desc_t, desc_t desc>
2 struct fs_hd_t {
3     inode_t inode_tbl[desc.n_inodes];
4     bitset<N_BLKES> free_set; /* N_BLKES is total number of available blks */
5
6     bool valid() const;
7     void format(); // format the file system
8
9     // find the inode for the file fn
10    inode_t &find(fn_t fn);
11
12    // find nblks unclaimed blks and assign them to the given buffer
13    void alloc_blks(blk_addr_t *buf, size_t nblks);
14
15    // reclaim the given blks
16    void dealloc_blks(blk_addr_t *buf, size_t nblks);
17 };

```

The file system backend is then a wrapper around the header, that provides methods for:

- **open** and **remove** files.
- **load** and **store** data.
- **mount** and **format** operations.

```

                                include/fs/fs_bck.h


---


1 template <typename desc_t, desc_t desc>
2 class fs_t
3 {
4     fs_hd_t fs_hd;
5
6 public:
7     void format();
8     void mount(); /* if the header is not valid, it also formats it */
9     void umount() const;
10

```

```

11 // Open a file using Linux like flags O_CREATE, O_WRITE, O_SHARED etc.
12 const inode_t *open(fn_t fn, size_t sz, uint32_t flags);
13
14 void remove(fn_t fn);
15
16 // Load the contents of the file to the given buffer
17 void load(const inode_t *inode, uint8_t *buf, size_t buf_sz) const;
18
19 // Store the contents of the given buffer to the file
20 void store(const inode_t *inode, uint8_t *buf, size_t buf_sz) const;
21
22 // Store the contents of the buffer at a particular offset.
23 void store(const inode_t *inode, size_t off, uint8_t *buf,
24           size_t buf_sz) const;
25 };

```

load and **store** map buffer contents to blocks, allowing files to have non-contiguous block allocations.

Currently a file's size cannot be modified. But it should be fairly simple to increase the size of a file, just by allocating more blocks to it.

6.2.2 File System Frontend

The frontend maintains a table of open files and maps them to runtime memory addresses. This mapping can be either private or shared:

- Private mapping: A file is mapped to a runtime buffer local to the process.
- Shared mapping: Multiple processes access the same runtime representation of the file, enabling inter-process communication (IPC).

This mechanism is inspired by the `mmap` function in Linux.

File operations occur in the following stages:

Opening a File When a process requests to open a file `fn`:

- The frontend retrieves the associated inode, creating one if it doesn't exist.
- It locates an available file descriptor in the open files table.
- A file handler is wrapped around the descriptor and returned to the process.

Mapping a File The mapping strategy depends on whether the `O_SHARED` flag is set during the `open` call:

- Shared mapping:
 - A buffer is allocated, and its address is stored in the file descriptor.
 - If a mapping already exists, it is reused.
 - All **load** and **store** operations use this shared buffer.

■ Private mapping:

- A buffer is allocated and stored in the file handler.
- Other processes cannot access this mapping.

Loading and Storing a File Once mapping is established, all **load** and **store** operations reference the appropriate buffer from either the file descriptor (shared mapping) or file handler (private mapping).

The file descriptor maintains:

- A pointer to the underlying inode.
- A reference count for read and write operations.
- Information on shared memory mappings.

include/fs/fs_impl.h

```

1 class fd_t
2 {
3     uint8_t w_cnt, r_cnt; /* read and write reference counter */
4
5 public:
6     fn_t fn;
7     mutable mutex<8> mtx = {"fd"}; /* file operations should be atomic */
8     const inode_t *inode; /* the underlying file */
9     mmap_unit_t mu; /* mapping information */
10
11 public:
12     // if this file descriptor refers the file fn, increse ref count
13     // otherwise mark this descriptor in use
14     void acquire(fn_t _fn, const inode_t *_inode, bool w_en);
15
16     // decrease ref count,
17     // if it was the last reference to the file, close this descriptor
18     void release(bool w_en);
19 };

```

The file handler is what the process interacts with. It holds a pointer to the file descriptor and provides methods for interacting with the file, ensuring smooth access and management.

include/fs/fs_impl.h

```

1 class file_t
2 {
3     fd_t *fd;
4     mmap_unit_t mu; /* private mapping information */
5     const bool is_shared; /* is the mapping shared? */
6     const bool w_en = false; /* have write permission? */
7
8 public:
9     // open the file in an RAII style, close it in the desctructor
10    file_t(fn_t fn, size_t _sz, uint32_t flags);
11
12    ~file_t();

```

```

13  void close();
14
15  // create runtime mapping
16  void *mmap(size_t sz);
17
18  // instead of allocating new memory, use the given memory address
19  void mmap(uint8_t *buf, size_t sz);
20
21  // remove the mapping
22  void unmap();
23
24  void load();
25  void store();
26  // store a chunk of size sz at offset off
27  void store(size_t off, size_t sz);
28 };

```

Finally, the top-level file system implementation maintains:

- An array of open files.
- An `fs_t` instance referring to the backend.
- Methods for interacting with the file system backend.

```

                                include/fs/fs_impl.h


---


1  template <desc_t desc>
2  class fs_impl_t
3  {
4      // the backend implementation
5      inline static fs_t fs;
6
7      // pool allocator providing memory for shared mapping
8      inline static allocator<alloc_heap, 4> pool;
9
10     // table of open files
11     inline static fd_t open_files[desc.n_open_files] = {};
12
13 public:
14     static file_t open(fn_t fn, size_t sz, uint32_t flags);
15     static void remove(fn_t fn);
16     static void mount();
17     static void format();
18 };

```

6.2.3 File system instantiation

So far, the file system has been an abstract concept implemented through a complex network of templates. To create actual file systems, we need to instantiate these templates with specific parameters. After defining the required parameters for both the flash and FRAM file systems, the instantiation process becomes straightforward:

```
include/fs/fs.h
```

```
1 // instantiate with the parameters for the flash fs
2 class flash : public fs_impl_t<_flash::desc> { };
3
4 // the same for the fram fs
5 class fram : public fs_impl_t<_fram::desc> { };
```

Now, we can use files as follows:

```
                                apollo/test.cpp
1 void file_rw_test()
2 {
3     fn_t fn = 10;
4
5     auto file = fram::open(fn, 16, O_WRITE | O_CREATE);
6     auto t = (char *)file.mmap(16);
7
8     const char *s = "It IS a string";
9     while (*s != '\0')
10         *t++ = *s++;
11     *t = '\0';
12
13     file.store();
14     // the file is automatically closed by the destructor
15 }
```

§7 Error Handling and Recovery

For a critical mission like the Apollo program, where astronauts' lives depended on flawless execution, thorough testing of the entire codebase was essential. However, no matter how well-tested a system is, some errors only show up randomly due to hardware faults or unforeseen corner cases. To handle such situations, the AGC's operating system had a robust error handling and recovery mechanism.

Because of the thorough testing during development, the AGC developers assumed that any error occurring at runtime was likely from some external source rather than a software bug. This made the age-old solution of "just rebooting it" a reasonable recovery strategy.

Of course, an unexpected restart during a critical operation could cause problems. To mitigate this, the AGC maintained a restart table, where processes could store recovery information, ensuring they could resume correctly after a reset.

Apollo-RTOS follows a similar strategy for error handling and recovery. Before explaining the implementation, we need to discuss how the system determines its current state.

7.1 Boot Levels

From an error-handling perspective, the device can be in one of four states:

- **FLASH** – The device has just been flashed. Everything is shiny and new, and all hardware has been reset.
- **POWER** – The device was unplugged and powered back on, resulting in a state similar to **FLASH**.
- **BOOT** – The reset button was pressed, causing a full restart, though some registers may retain their values.
- **RESET** – A software reset occurred. RAM contents are retained, but execution restarts from the beginning.

According to the Nordic nRF51 technical reference manual [5], RAM contents are retained in a soft reset, but may be corrupted in other forms of reset.

Now, suppose an error occurs. There are three ways to handle it:

- Terminate the faulty process and let the system continue.
- Trigger a soft reset, rebooting the entire system automatically.
- Halt and wait for manual intervention, requiring a physical reset.

A power outage is a more extreme case. In theory, once the device loses power, the mission has already failed. But since Apollo-RTOS isn't responsible for landing a spacecraft, we take a more lenient approach: when power is restored, the system attempts to resume major tasks that were running before the outage.

7.1.1 How to Detect Boot Stages

To distinguish between these boot stages, we use the retention properties of different memory regions:

- **FLASH** – Flash memory is completely overwritten when reflashed. If a magic value at a specific flash address is missing, we know the system was just flashed.
- **RESET** – Since RAM is retained during a soft reset, we store a magic number at a particular RAM address. If it's still there after a reboot, we've had a soft reset.
- **POWER** – At first power-on, the `POWER.RESETREAS` register is set to . This register tracks reset causes, so technically, we could use it to detect soft resets as well—but we prefer the magic number solution.
- **BOOT** – If none of the above conditions apply, we assume we're in a normal boot state.

All of this logic is encapsulated in the `boot` namespace, providing a simple way to determine the system's current state and react accordingly.

```
include/code/boot.h
```

```

1 namespace boot
2 {
3 boot_lev_t lev(); /* tells us the current boot level */
4 void init(); /* discovers what the current level is */
5 } // namespace boot
```

As a demonstration, here's what Apollo-RTOS prints on boot:

```

boot level: flash
boot stat: n_flash = 395, n_boot = 22, n_power = 19, n_reset = 114
```

This output shows the current boot level (flash) and the number of times the system has gone through different reset modes since the last FRAM file system format (one week ago).

7.2 Error handling

The scary part about not having a memory protection unit for the RAM is that a process can virtually rewrite any memory address. So in situations where we would really like to get a `segfault`, we will get silence, only for the system to crash horribly long time later.

So it's up to the programmer to ensure that the programs are free of bugs. The best way to ensure that is by having lots of checks throughout the code that checks parameters, results of calculations, and anything that could go wrong.

The C-header `<assert>` provides a macro `assert` that allows us to check that something holds, throwing an error if it doesn't. Since we don't have a mechanism for throwing errors in our microcontroller, we have to implement our own `assert`.

Before that, let's talk about how we debug things in a microcontroller. The only true answer to that question is: by printing stuff. So we need an implementation of `printf`. This `printf` implementation in Apollo-RTOS uses a buffer to send characters over the serial bus

asynchronously. So we also provide a blocking implementation in `kprintf`. We then can implement a convenience function `debug`, which only outputs stuff if the debug level is high enough.

```

                                include/utility/debug.h
1  template <debug_t level, typename... Args>
2  void debug(const char *fmt, Args... args)
3  {
4      if constexpr (level <= DEBUG_LEV) {
5          if constexpr (level <= ERROR)
6              kprintf(fmt, args...);
7          else
8              printf(fmt, args...);
9      }
10 }
```

With that, we can now implement a lightweight `assert` method. We use the macro as `assert(expr, lev)` expecting to cause a reset at the level `lev` if the expression `expr` evaluates to `false`. `lev` can be one of the three values:

- `H_RESET`: causes a hardware reset if the assertion is false,
- `S_RESET`: causes a soft reset,
- `TERM`: terminates the current process.

```

                                include/utility/debug.h
1  template <auto lev, typename... Args>
2  void __assert(bool ex, Args... args)
3  {
4      if (ex)
5          return; // everything under control
6
7      if constexpr (lev == H_RESET) {
8          if constexpr (sizeof...(args) != 0)
9              /* print debug information */
10
11          return trigger_hardfault(); // halt for manual intervention
12      }
13
14      if constexpr (lev == S_RESET)
15          return trigger_reset(); // cause a soft reset
16
17      if constexpr (lev == TERM)
18          return trigger_termination(); // terminate the current process
19  }
20
21 #define assert(EX, LEV, ...) __assert<LEV>((EX), ##__VA_ARGS__)
```

To halt for manual intervention, we trigger a hardfault, which calls the `hardfault_handler`. In the hardfault, the system prints some debug information and waits in a loop for either a soft reset caused by pressing CTRL(d), or a pin reset. Interrupts are also automatically disabled in the hardfault handler.

```

src/core/hardfault.cpp
1 extern "C" void hardfault_handler()
2 {
3     // ... print the pc and lr registers from the faulted stack frame
4
5     while (true) {
6         switch (serial::getc()) {
7             case CTRL('d'):
8                 trigger_reset(); // soft reset can be raised from a hardfault
9                 break;
10
11             case '?':
12                 // dump debug information
13                 break;
14         }
15     }
16 }

```

7.3 Recovery

Now that we've handled the errors and caused an appropriate reset, we can talk about how the recovery mechanism actually works.

The recovery mechanism keeps a table where recovery information is stored. Upon entering a critical section that needs to be protected against a reset, a process or a task added an entry to this table that describes what to do after a restart if a reset were to happen while the process or the task was still in the critical section.

To be able to recover from a power off reset, this recovery table is stored in a file in the FRAM file system. Everytime a process or a task updates an entry in the table, the relevant portion of the table is updated in the FRAM as well.

Recovery of processes and tasks are handled differently. So we will explain them separately.

7.3.1 Recovering a process

The only meaningful recovery mechanism for a process is to just restart the process. But the process needs to know which state it was when it was interrupted by the reset, so there's a 20B long unstructured storage in the recovery table entry where the process can store information about it's last running state. When the process is restarted by the recovery mechanism, this data is passed to the process as the parameter. The process should then decide how to proceed from there.

A recovery entry for a process looks like this (the actual implementation differs in structure, but has the same fields). The **recover** function is called when the system restarts, and notices that this entry is occupied by a process.

```

src/core/recover.cpp

```

```

1 struct alignas(uint32_t) entry_proc_t {
2     const proc_def_t *proc_def;
3     lev_t lev = NONE; // discussed later
4
5     uint8_t data[N_REC_DATA];
6     uint8_t data_sz;
7
8     void recover(lev_t curr_lev)
9     {
10         if (lev < curr_lev)
11             return;
12
13         uint8_t *copied_data = nullptr
14         if (data_sz > 0) {
15             copied_data = (uint8_t *)kmem::kmallocc(data_sz);
16             memcpy(copied_data, data, data_sz);
17         }
18
19         sched::reg_proc(proc_def, copied_data);
20     }
21 }

```

Notice the field `lev_t lev` in the entry. There can be three levels of recovery:

- **NONE**: the default, no process is occupying this entry, so nothing is recovered from this entry.
- **RESET**: the process occupying this entry should be restarted after hardware and soft reset.
- **POWER_OFF**: the process in this entry should also be started after a power on reset. This is the highest level, so a process with recovery level **POWER_OFF** is also restarted during a hardware or soft reset.

When a process exits its critical section, it should remove or change it's recovery policy. It does so by setting the entry level to **NONE**. The entry then can be reused by other processes to set their recovery policy. This “acquire” and then “release” sequence sounds like it could be made into RAIL, and that's what we have:

```

include/core/recover.h
1 struct guard_proc {
2 private:
3     size_t recovery_table_entry;
4
5 public:
6     // acquire an entry, set the current process' recovery level to lev,
7     // and store the data
8     guard_proc(lev_t lev, const uint8_t *data, size_t data_sz);
9
10    // convenience
11    template <typename T>
12    guard_proc(lev_t lev, const T &data)
13        : guard_proc(lev, (const uint8_t *)&data, sizeof(T)) {}
14

```

```

15 // release the entry by setting the recovery level to NONE
16 ~guard_proc();
17 };

```

With this setup, we can use it in practice just by writing

```

1 { /* start of process */
2   ...
3   { /* start of critical section */
4     int i = /* whatever */;
5     recover::guard_proc guard{POWER_OFF, i};
6
7     // ... do critical stuff
8
9   } /* recovery table entry is automatically released */
10  ...
11 }

```

7.3.2 Recovering a task

Recovery of a task is similar to recovering a process, but instead of restarting the original task, the task can assign any arbitrary function to be run during startup.

src/core/recover.cpp

```

1 struct alignas(uint32_t) entry_task_t {
2   time_t interval = 0;
3   runnable_t task = nullptr;
4   lev_t lev = NONE; // discussed later
5
6   uint8_t data[N_REC_DATA];
7   uint8_t data_sz;
8
9   void recover(lev_t curr_lev)
10  {
11    if (lev < curr_lev)
12      return;
13
14    uint8_t *copied_data = /* copy data here */;
15
16    if (interval == 0)
17      task(copied_data);
18    else
19      waitlist::reg("recover", interval, task, copied_data);
20  }
21 }

```

And similar to **guard_proc** above, there is a RAII class **guard_task** that acquires a recovery entry for a task at the start of a critical section and releases it and the end of the section automatically:

apollo/test.cpp

```

1 void reset_task_test()

```

```

2 {
3   { /* start of critical section */
4     auto cleanup = [](void *p) { /* whatever */ };
5     recover::guard_task guard{RESET, cleanup, 0, /*data to be passed*/};
6     // run cleanup immediately after restart
7
8     // ... do stuff
9   }
10
11  /* no recovery after task has completed the critical section */
12 }

```

7.4 Recovery at the startup

So far we have seen how processes and tasks declare the policy for recovering them from a reset. Where in the boot sequence is this recovery done? To understand that, we need to understand how the boot sequence in a microcontroller like `micro:bit` works.

When the `micro:bit` starts, it reads off the stack address from the address `0x0` and loads it to the `sp` register. It then starts executing the code from the instruction stored at address `0x4`. It's apparent when we look at the vector table:

```

src/vector_table.c
1 __attribute__((section(".vectors"))) void *__vectors[] = {
2   __stack, // Initial Main Stack Pointer (MSP) value
3   __reset, // Reset Handler: Entry point after power-up or reset
4
5   ... // interrupt handlers
6 }

```

This vector table lists all the interrupt handlers as well. This table is placed at the address `0x0` by the linker.

So it is this `__reset` function where our execution begins. If we have a look at the `__reset` function,

```

src/startup.cpp
1 extern "C" void __reset(void)
2 {
3   // ... activate the clock
4   // ... setup memory protection for the code area
5   // ... initialize data sections like .data and .bss
6   // ... initialization for various modules and drivers
7
8   sched::init();
9 }

```

It's the line 8 that actually sets up all the startup processes, and starts execution from the idle process as discussed in § 5.5. After registering the idle process, `sched::init` must set up the rest of the processes. This is where it calls the two functions:

 src/core/sched.cpp

```

1 void sched::init()
2 {
3     // setup init_proc
4
5     setup_startups();
6     recover::init();
7
8     timer::init(); // start the timer
9
10    // jump to init_proc
11 }

```

`setup_startups` sets up processes that MUST run at the beginning no matter what. Currently there are two processes `shell` and `display` that are run as part of the startups. `shell` implements the Linux like command prompt, and `display` handles displaying images on the LEDs.

We're more interested in `recover::init`. This function first checks what the boot level is, and whether this startup was due to a soft reset or a power on reset.

- If this startup was due to the device being freshly flashed, or if it was due to the `pin reset`, then `recover::init` setups up another set of startup processes. For example, it sets up a driver process called `accel` that constantly read from the accelerometer and stores the data in a file that other processes can access.
- However, if the startup was due to a soft reset or a power on reset, then the recovery table is scanned for processes and tasks needing to be recovered.

 src/core/recover.cpp

```

1 void recover::init()
2 {
3     auto file = /* open and load the recovery file containing the table */
4     auto &tbl = /* reference the recovery table */
5
6     auto boot_lev = boot::lev();
7     if (boot_lev == BOOT || boot_lev == FLASH) {
8         tbl.reset(); // reset the table
9         sched::setup_procs(); // set up processes that runs at startup
10    } else {
11        auto reset_lev = boot_lev == RESET ? RESET : POWER_OFF;
12        tbl.recover(reset_lev);
13        // recover from entries that have policy to recover at this level
14        // this also resets the entry
15    }
16
17    file.store(); // finally store the now reset recover table
18 }

```

With that set up, the execution can continue smoothly.

§8 Drivers

Drivers provide methods for interacting with different hardware components on a device. Typically, hardware components expose registers that control various functionalities. The role of a driver is to abstract these low-level register interactions and provide a user-friendly interface.

8.1 Timers

The Nordic nRF51 board on the `micro:bit` features three timers:

- `TIMER0` – A 32-bit timer.
- `TIMER1` and `TIMER2` – 16-bit timers.

Apollo-RTOS configures the two 16-bit timers as 1MHz timers, generating an interrupt every 1s. These timers serve distinct purposes:

- `TIMER1` – Maintains the system clock.
- `TIMER2` – Used for profiling.

The initialization function for both timers is straightforward, primarily involving writing values to specific registers in the correct order. This implementation was inspired by Spivey [6].

The interrupt handling for `TIMER1` (the system timer) is more intricate and will be discussed next.

```
src/drivers/timer.cpp
1 volatile time_t MILLIS = 0;
2
3 extern "C" void timer1_handler(void)
4 {
5     // increment MILLIS
6
7     sched::assert_stack(); // check for stack overflow
8
9     // run the waitlist tasks every (update_interval)us
10    if (MILLIS % waitlist::update_interval == 0) {
11        intr_guard guard;
12
13        // setup temporary stk to run waitlist tasks
14        auto prev_stk = (uint8_t *)get_msp();
15        set_msp(/* temporary stack */);
16
17        waitlist::run();
18
19        // restore original stack
20        set_msp(prev_stk);
21    }
```

22 }

As mentioned earlier in § 5.6, at regular intervals (every `update_interval` μ s), the waitlist is checked for any pending tasks. Since the timer interrupt can occur at any point in execution, there is a risk of stack overflow. To mitigate this, waitlist tasks are executed on a separate stack. This is why a temporary stack is created and set up in lines 14-15.

8.1.1 Profiling

This is an experimental feature currently under development. The idea is to use `TIMER2` to track CPU time consumed by each process and function. At present, it performs reasonably well for profiling processes but struggles with function-level profiling.

For processes, users can type `trace` in the shell to display process statuses along with their CPU usage:

```
curr_proc: trace
| 0. idle      : (idle),    [runnable], 38.938217%
| 1. shell     : (high),    [asleep],   0.18498%
| 2. display   : (low),     [runnable], 48.945616%
| 3. accel_bg  : (high),    [runnable],  0.18498%
| 4. test      : (low),     [asleep],   1.609323%
| 5. trace     : (medium),  [runnable], 14.95450%
```

As seen in the output, most CPU time is currently spent in the `idle` and `display` processes—mainly because I haven’t written a more demanding workload. A significant portion is also used for printing this information over the serial bus, reaffirming the well-known overhead of I/O operations.

This behavior is implemented in the `timer2_handler`:

```
src/drivers/timer.cpp
1 volatile time_t debug_ticks = 0;
2
3 extern "C" void timer2_handler(void)
4 {
5     // increment debug_ticks
6
7     sched::tick(); // increment tick for the current process
8     profile::tick(); // increment tick for the currently running function
9     // but only do this for functions that have profiling enabled
10 }
```

8.2 Serial

The serial driver manages the UART connection. The `serial::init()` function initializes the UART device and enables interrupts. The `serial` namespace provides three core functions:

- **void busy_putc(char):** A **putc** function that disables interrupts and busy-waits until the character is sent.
- **char getc():** Waits for a character when interrupts are disabled.
- **void intr_putc(char):** A buffered **putc** implementation that uses interrupts.

The **busy_putc** and **getc** functions are straightforward: they send or receive a character and wait for the device to become ready. The **intr_putc** function, however, is more interesting and is partially inspired by the implementation by Spivey [6].

```
src/drivers/serial.cpp
```

```

1 volatile int txidle; /* Whether UART is idle */
2 circular_buffer<char, NBUF> buf;
3
4 void intr_putc(char ch)
5 {
6     // if the buffer is full
7     while (buf.size() == NBUF)
8         pause(); // enter standby state waiting for an interrupt
9
10    // if the UART is idle, send the character immediately
11    if (txidle) {
12        UART.TXD = ch;
13        txidle = 0; // mark the UART busy
14    }
15    // otherwise enqueue the character
16    else
17        buf.enqueue(ch);
18 }
```

With this setup, when functions like **printf** send multiple characters, usually only the first character is transmitted immediately. The remaining characters are placed into a circular buffer, and it becomes the UART interrupt handler's responsibility to send them asynchronously.

```
src/drivers/serial.cpp
```

```

1 extern "C" void uart_handler(void)
2 {
3     // ... handle the receive path here
4
5     // UART finished transmission
6     if (UART.TXDRDY) {
7         UART.TXDRDY = 0;
8         if (buf.empty()) // no more characters to send
9             txidle = 1; // mark UART idle
10        else
11            UART.TXD = buf.dequeue(); // otherwise send the next character
12    }
13
14    // ... clear and re-enable interrupts
15 }
```

8.2.1 Listening on the UART

In Apollo-RTOS, all incoming UART characters are, by default, sent to the `shell` process. This creates the illusion that the entire operating system is simply the shell, allowing user interaction through commands.

However, some processes may need to momentarily take control of input, similar to starting a Python shell within a Bash shell. To enable this, the `serial` driver maintains a stack of “listeners” that process incoming characters. The UART handler sends each character from the top of the listener stack downward. If a listener “consumes” the character, it is not passed to any other listeners.

To explain what I mean:

```

                                include/drivers/serial.h
1  using listener_t = bool (*)(char);
2
3  void register_listener(listener_t);
4  void unregister_listener();

                                src/drivers/serial.cpp
1  class {
2      listener_t stack[16] = {nullptr};
3      size_t idx = 0;
4
5  public:
6      void push(listener_t listener) { stack[idx++] = listener; }
7      void pop() { idx--; }
8
9      void send(char c)
10     {
11         auto i = idx;
12         while (i > 0) {
13             if (stack[--i](c))
14                 break; // if consumed, don't send c to the rest of the listeners
15         }
16     }
17 } listeners;
18
19 void register_listener(listener_t listener) { listeners.push(listener); }
20 void unregister_listener() { listeners.pop(); }

```

Finally in the UART handler, all incoming characters get sent to listeners

```

                                src/drivers/serial.cpp
1  extern "C" void uart_handler(void)
2  {
3      if (UART.RXDRDY) {
4          listeners.send(UART.RXD); // send the character to the listeners
5          UART.RXDRDY = 0;
6      }
7

```

```

8 // ... the rest
9 }

```

Of course, this registering and unregistering pattern can be put into an RAII class:

```

                                include/drivers/serial.h
1 struct listener_guard {
2     listener_guard(listener_t listener) { register_listener(listener); }
3     ~listener_guard() { unregister_listener(); }
4 };

```

8.3 I2C

The I2C master on the Nordic nRF51 enables communication with devices connected to the `micro:bit` via the I2C bus. It operates at frequencies of 100KHz or 400KHz. For this project, we use the 100KHz configuration provided by Spivey [6].

Apollo-RTOS includes two I2C driver implementations: synchronous and asynchronous. The synchronous implementation is straightforward, with all read and write operations busy-waiting until completion. This implementation is used before the scheduler is initialized.

Once the scheduler is active, the asynchronous implementation takes over. In this mode, the I2C backend uses a mutex to synchronize requests from different processes. From the process's perspective, each I2C operation appears atomic. However, due to the nature of I2C communication, sending multiple bytes triggers multiple interrupts. To maintain the illusion of atomicity, the process is put to sleep while communication is in progress.

Below is a brief explanation of how the I2C master communicates with slave devices:

- First, the device address is written to the `I2C0.ADDRESS` register. This address is unique to each device and ranges between `0x03` and `0x77`.
- Next, the bytes constituting the "command" are written. After each byte, an acknowledgment interrupt is raised, signaling the system to send the next byte.
- After all command bytes are sent, the process differs for read and write operations:
 - For write operations, bytes are sent sequentially, with acknowledgment interrupts confirming each transmission.
 - For read operations:
 - The `STARTRX` register is set to 1.
 - If this is the last byte to be read, the `STOP` bit in the `SHORTS` register is set.
 - Otherwise, the `SUSPEND` bit is set.
 - The device writes the requested data to the `RXD` register and raises an interrupt.
 - The interrupt is detected, and the data is read from the `RXD` register.
 - If more bytes are to be read, the `RESUME` register is set to 1.

To manage this process, we implement a state machine with four states:

- **W_CMD**: Writing command bytes.
- **TX_DATA**: Transmitting data.
- **RX_DATA**: Receiving data.
- **NACK**: Finalizing the operation and performing cleanup.

The communication begins in the **xfer** method:

```

src/drivers/i2c_async.h
1 template <bool is_read>
2 int xfer(uint8_t addr,
3         uint8_t *cmd, size_t cmd_sz,
4         uint8_t *buf, size_t buf_sz)
5 {
6     // acquire lock before continuing
7     mtx.lock();
8
9     // ... copy the buffer addresses and buffer sizes
10
11    // initial state is W_CMD
12    state.stage = stage_t::W_CMD;
13    I2C0.ADDRESS = addr;
14
15    // start transmission and send the first byte
16    I2C0.STARTTX = 1;
17    I2C0.TXD = *cmd;
18
19    // put the current process to sleep until woken up after completion
20    sched::wait(intr_chan);
21
22    // ... report any error that might've occurred
23 }
```

This method just starts the transmission, and puts the process to sleep. The real magic (or nightmare if implemented incorrectly) happens in the interrupt handler:

```

src/drivers/i2c_async.h
1 void handler(void)
2 {
3     intr_guard guard{I2C0_IRQ};
4
5     if (I2C0.ERROR)
6         // ... if there was an error, cancel the communication
7
8     else if (state.stage == stage_t::W_CMD) {
9         // ... receive acknowledgement
10
11         // still writing the command
12         if (state.cmd_idx < state.cmd_sz)
13             // ... send next byte
14
15         // no data to send or receive, so stop
16         else if (state.data_sz == 0)
```

```

17     // ... stop
18
19     // if we're writing
20     else if (state.mode == mode_t::WRITE) {
21         state.stage = stage_t::TX_DATA;
22         // ... send the first data byte
23     }
24
25     // we're reading
26     else {
27         state.stage = stage_t::RX_DATA;
28         // ... set the SHORTS register and STARTRX = 1
29     }
30 }
31
32 else if (state.stage == stage_t::TX_DATA) {
33     // ... receive acknowledgement
34
35     // finished writing
36     if (state.data_idx == state.data_sz)
37         // ... stop
38
39     // more bytes to write
40     else {
41         state.stage = stage_t::TX_DATA;
42         // ... send the next byte
43     }
44 }
45
46 else if (state.stage == stage_t::RX_DATA) {
47     // ... receive acknowledgement
48
49     // ... copy byte from I2C0.RXD
50
51     // finished reading, move to NACK
52     if (state.data_idx == state.data_sz)
53         state.stage = stage_t::NACK;
54
55     // continue reading
56     else {
57         state.stage = stage_t::RX_DATA;
58         // ... set the SHORTS register and RESUME = 1
59     }
60 }
61
62 else if (state.stage == stage_t::NACK) {
63     // ... receive acknowledgement
64
65     mtx.unlock(); // release the lock
66     sched::notify(intr_chan); // then wake up the waiting process
67 }
68 }

```

With this implementation, multiple processes can communicate over the I2C bus without

conflicts. All requests are serialized using the `mutex`, ensuring orderly access to the bus. Additionally, while one process is communicating over the I2C bus, it does not monopolize the CPU. Instead, it allows other processes to continue executing, improving overall system efficiency.

8.4 Display

8.5 Display

The 5×5 LED matrix is arranged into three columns in a counter-intuitive format. To control these LEDs, we utilized the convenience macros provided by Spivey [6]. The LEDs are managed through GPIO registers and require constant updates to display smooth animations. To achieve this, the `display` driver is implemented as a low-priority process, with its priority set just above the `idle_proc`.

The implementation is straightforward:

```
src/drivers/display.cpp
1 image_t image;
2
3 /* definition of display */
4 {
5     GPIO.DIR = LED_MASK;
6     reset(); // clear the image out at the beginning
7
8     while (true) {
9         int n = 0;
10        // display the three rows of the image, and wait for a little
11        while (n < 3) {
12            GPIO.OUT = image[n++];
13            delay_loop(5000);
14        }
15        sched::sleep(10);
16    }
17 }
```

A method named `set_image` is provided to allow processes to update the global variable `image` with the desired display image. The display driver continuously renders the image stored in this variable.

As discussed earlier in § 7.4, this process is always started during system initialization.

§9 Shell and Shell Commands

Apollo-RTOS provides a Linux-like `shell` that listens to inputs on the serial bus, and once a full command has been sent out, it schedules a the corresponding process.

The `shell` is a high priority process, that spends most of its time asleep. At startup, it registers a `listener` that wakes up `shell` when it receives a `'\r'` or `'\n'` character. The `shell` then becomes the highest priority process and processes the command as soon as it is allowed to run.

When processing the command, `shell` allocates some memory to copy the command line arguments, and passess it to the process it schedules. It then clears out it's internal buffer, and goes to sleep again.

```
src/core/shell.cpp
1 chan_t<1> chan; // channel of length 1 for the shell to wait on
2 args_buffer_t buf; // buffer where the command is accumulated
3
4 /* definition of shell */
5 {
6     serial::register_listener(listener);
7
8     while ((volatile bool)true) {
9         buf.reset();
10        sched::wait(chan); // wait until the listener wakes it up
11
12        const char *cmd = /* extract the command string */;
13        proc_def_t *cmd_def = /* find corresponding process defintiion */;
14
15        if (cmd_def == nullptr)
16            continue; // ignore this command
17
18        auto param = /* allocate memory and copy the arguments */;
19        sched::reg_proc(cmd_def, param);
20
21        // ... rinse and repeat
22    }
23 }
```

And the `listener` is implemented as such:

```
src/core/shell.cpp
1 bool listener(char c)
2 {
3     // ... handle some special characters
4
5     // print the character back
6     serial::intr_putc(c);
7
8     // if the character is a normal character, append it to the buffer
9     if (c >= 32 && c < 127) {
10        buf.push(c);
```

```

11     return true;
12 }
13
14 // if the CR or Return character is received, wake up the shell process
15 if (c == '\r' || c == '\n') {
16     sched::notify(chan);
17     return true;
18 }
19
20 // any unhandled character resets the buffer
21 printf("'%d'\r\n", c);
22 buf.reset();
23
24 // this listener processes all characters,
25 // and is the last listener in the listeners stack
26 return true;
27 }

```

9.1 Notable Shell Commands

Currently there are a few shell commands that are useful to demonstrate different aspects of the system. Here are some of them:

trace This command produces a dump of the current processes, waitlist tasks, current open files etc:

```

trace

boot level: reset
curr_proc: trace
| 0. idle : (idle), [runnable], 44.88736%
| 1. shell : (high), [asleep], 0.0%
| 2. display : (low), [runnable], 54.258086%
| 3. accel_bg : (high), [runnable], 0.31%
| 4. test : (low), [asleep], 1.652978%
| 5. trace : (medium), [runnable], 0.2881%
waitlist:
| test
| interval: 48
fram fs:
| free_blks: 244, free: 31232, in use: 1536
| [O] 0: size = 1024, type = bin, blks: 1, 2, 3, 4, 5, 6, 7, 8
| [C] 2: size = 128, type = bin, blks: 0
| [O] 5: size = 128, type = bin, blks: 9

```

echo Does what the name suggests: echoes the argument:

```

echo hello world
>> hello world

```

accel Starts a small compass like program on the LEDs that always point an arrow towards the ground:

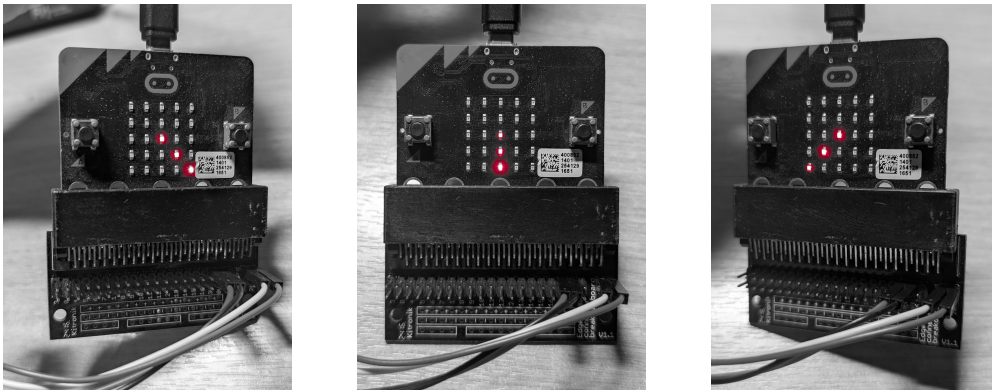


Figure 9.1: Result of `accl`

calc Starts a sub-shell like command that is “supposed” to implement a calculator. Right now, it just takes over the inputs, and echoes the arguments it receives.

§10 Evaluation and Testing

Our goal was to write an automated testing framework that'll allow one to write unit, sub-system and bevioural tests that are run in isolation of each other. Several factors made this endeavor quite involved:

- The monolithic design of the operating system makes it difficult to verify and test each module in isolation. As a result, mistakes in one module can (and it did) manifest in seemingly irrelevant places, making it very difficult to track down the bugs.
- The tests need to be run on the actual microcontroller instead of an emulator. The problem with that is that copying a binary to the device is a rather slow process. So to write an automated testing system, all the tests need to be shipped together and copied to the device in a singular binary. So it is upto the testing framework to run the tests in isolation.
- Unit tests that verify core functionality like `malloc` or `files` need to be run independently of the scheduler. So these tests must be run before the scheduler is initialized.

Tests for the scheduler and other behavioral tests that verify collaboration between different modules need to be run with the scheduler initialized.

Furthermore, each individual test must be run in full isolation, so that misbehavior in one module might not effect the behaviour of another.

With these constraints in mind, we designed a testing framework. The design is slightly inspired by the Boost.Test library [1], with macros that automatically register tests. We first give a brief overview of the testing framework before discussing the tests currently available for the OS.

10.1 Testing Framework

Each “test” is described by a test descriptor of the format:

```

1 struct test_t {
2   const char *name;           // name of the test
3   bool (*func)(void);        // test function to be run
4   bool *passed;               // storage for the result of the test
5 };

```

There are two sections in the binary file to store test descriptors: `.unit_tests` and `.sys_tests`. Unit tests meant to be run before the scheduler is initialized are placed in the former section, and tests to be run with the scheduler are placed in the latter.

There are two macros available in the `include/core/test.h` header file:

```

1 #define TEST(name) ...
2 #define SYS_TEST(name) ...

```

which are used as follows:

```

1 namespace TEST_SUITE(test_suite) {

```

```

2 TEST(test_name) {
3     ...
4     return true;
5 }
6 }

```

The macro **TEST** defines a test descriptor for the test, places it in the `.unit_tests` section. Similarly for **SYS_TEST** macro.

During startup, before the scheduler is initialized, the tests are run. The algorithm is as follows:

- During startup and before scheduler initialization, `test::run()` method is called.
- If the boot stage is not **RESET** (defined in § 7.1.1), `test::run()` initializes the two sections `.unit_tests`, `.sys_tests`.
- It then starts running the tests in the `.unit_tests` section. For each test in this section,
 - It calls the test function, storing the result in address stored in `passed`.
 - The memory pointed to by `passed` is in the `.recovery` section which is not erased during soft resets.
 - `test::run()` then triggers a soft-reset, causing the whole system to restart. The system then follows this same path here, and carries on to the next test.
- After running all the `.unit_tests`, `test::run()` then schedules a process that runs the tests in `.sys_tests` similarly to above.
- It then hands control back to the startup method, which initializes the scheduler, and continues to running the system tests.
- To run one system tests, the whole system is restarted, a process is scheduled, then when that process is run, it runs the system test, stores the result, and triggers a software reset, moving to the next system test.

This reset mechanism before running each test ensures that an error state left by an earlier test does not effect subsequent tests, and also that each test is run in isolation. One caveat is that true isolation is not fully achieved, because if there's an issue with the recovery mechanism that leaves erroneous states uncleared, it will effect further testing.

10.2 Evaluation of Apollo-RTOS

With the time constraints in mind, only the most crucial aspects of the system have been tested. Below is a brief summary of unit and subsystem tests:

Memory allocator ■ The allocator

Scheduler ■ hello 1

 ■ hello 2

§11 Conclusion

This project aimed to develop an operating system inspired by the Apollo Guidance Computer (AGC), focusing on its scheduling and error-handling algorithms. The result, Apollo-RTOS, reimagines the core ideas of the AGC with modern concepts, demonstrating that the AGC's algorithms were ahead of their time and remain relevant for efficient operating system design.

The strength of these algorithms lies in their simplicity, a theme central to Apollo-RTOS's implementation. This simplicity makes the system easy to understand, explain, and verify compared to more complex designs. Apollo-RTOS shows that even with the `micro:bit`'s limited resources—a 16MHz CPU and 16KB of RAM—careful design can achieve significant functionality. The system can concurrently run 16 processes, serialize their execution deterministically, and handle errors gracefully, making it a strong candidate for real-world applications.

However, there is still room for improvement. Testing in this project was limited, and it was impossible to explore every possible combination of events within the given timeframe. Many hardware features were not tested under adverse conditions, which is crucial before real-world deployment. Additionally, the implementation is not fully optimized. Memory and CPU usage were managed leniently, without strict restrictions on execution time. For an RTOS, strict guarantees on upper bounds for each operation are essential, and Apollo-RTOS currently lacks these guarantees or has not yet been tested for them.

In terms of features, Apollo-RTOS does not yet support all hardware components available on the `micro:bit`. Notably, it lacks drivers for the SPI bus, watchdog timer, sensors, radio device, and others. Some limitations also stem from the hardware constraints of the `micro:bit` v1. For example, the Cortex M0 processor lacks a floating-point unit (FPU), requiring software libraries to handle floating-point arithmetic. Integrating such a library remains a task for future work.

Despite these limitations, Apollo-RTOS highlights how historical concepts can still be applied in modern contexts. It also demonstrates the feasibility of creating a general purpose, lightweight, efficient, and reliable operating system for resource-constrained micro-controllers, opening new possibilities for real-world applications.

References

- [1] Boost.test - 1.82.0, 2022. URL https://www.boost.org/doc/libs/1_82_0/libs/test/doc/html/index.html.
- [2] lady ada and Michael Schroeder. Datasheet for the mb85rc256vpf fram chip, 05 2014. URL <https://cdn-learn.adafruit.com/assets/assets/000/043/904/original/MB85RC256V-DS501-00017-3v0-E.pdf?1500009796>.
- [3] Frank O'Brien. *The Apollo Guidance Computer: Architecture and Operation*. Springer Science & Business Media, 01 2010. ISBN 978-1-4419-0876-6. doi: 10.1007/978-1-4419-0877-3.
- [4] Nordic Semiconductor. nrf51822 product specification v3.1, 2014. URL <https://spivey.oriel.ox.ac.uk/bmwiki/images/7/7e/NRF51822-product-spec.pdf>.
- [5] Nordic Semiconductor. nrf51 series reference manual, 2014. URL <https://spivey.oriel.ox.ac.uk/bmwiki/images/3/34/NRF51822-ref-manual.pdf>.
- [6] Mike Spivey. micro:bian. <https://github.com/Spivoxity/microbian>, 2021.
- [7] Stephen Witt. Apollo 11: Mission out of control, 06 2019. URL <https://www.wired.com/story/apollo-11-mission-out-of-control/>.

§A Codebase Structure

The codebase follows a standard structure, with headers residing in the `include/` directory and implementations in `src/`. These directories are further divided into subdirectories to group related files.

A brief overview of the key directories and files is as follows:

■ **include/**: Contains headers for core functionality.

• **core/**: Core system components.

- `hardware.h` — Hardware register descriptions (adapted from Spivey [6]).
- `types.h` — Type aliases and constants shared across the codebase.
- `irq.h` — Header-only interrupt handling utilities.
- `boot.h` — Functions for retrieving the current boot level.
- `sched.h` — Scheduler interface.
- `waitlist.h` — Methods for registering waitlist tasks.
- `memory.h` — Memory management utilities.
- `recover.h` — Recovery mechanisms for processes and tasks.
- `signal.h` — Methods for inter-process signaling.

• **drivers/**: Hardware drivers.

- `timer.h` — Provides system time (`now`) and profiling (`debug_ticks`).
- `display.h` — Convenience functions for displaying images on the LED matrix.
- `i2c.h` — Methods for I2C communication.
- `accel.h` — Reads accelerometer data.
- `gpio.h` — GPIO utility functions.
- `serial.h` — Serial I/O functions, including an interface for redirecting input from the shell to other processes.

• **utility/**: General-purpose utilities.

- `allocator.h` — Core memory pool allocator used by different `malloc` implementations.
- `format.h` — Provides `printf`-style formatting functions.
- `debug.h` — Conditional debugging output based on verbosity levels; also includes the `assert` macro.
- `mutex.h` — Lightweight mutex implementation using wait queues and `sleep/wakeup` from `sched.h`.
- `profile.h` — Macros for enabling function and process profiling.
- `args.h` — Data structures for passing command-line arguments to processes.
- `bitset.h` — Bitset implementation.

- `circular_buffer.h` — Circular buffer implementation.
- **fs/**: Header-only file system implementation (detailed later).
 - `fs_bck.h` — Backend for reading/writing device storage.
 - `fs_impl.h` — Manages open files, runtime memory maps, and file access.
 - `fs_desc.h` — Type definitions for template-based file system operations.
 - `fs.h` — Top-level file system interface, enabling separate instances for FLASH and FRAM storage.
- **src/**: Mirrors `include/`, implementing the corresponding functionality.
 - `vector_table.c` — Defines the interrupt vector table.
 - `mpx.s` — Handles context switching logic.
 - `device.ld` — Linker script defining memory layout.
 - `startup.cpp` — Initializes data sections and sets up core system modules.
- **apollo/**: Defines processes and built-in shell commands for Apollo-RTOS.
- **tests/**: Contains test cases demonstrating various aspects of the system.

=====					
Language	Files	Lines	Code	Comments	Blanks
=====					
Assembly	1	98	55	28	15
C	1	114	83	11	20
C++ Header	31	3402	2646	120	636
C++	27	2694	2128	75	491
=====					
Total	60	6308	4912	234	1162
=====					